DAM
Desarrollo de Aplicaciones Multiplataforma
2º Curso

AD
Acceso a Datos

UD 2
Manejo de Ficheros
Parte 1

IES BALMIS
Dpto Informática
Curso 2026-2027
Versión 1 (09/2026)

UD2 – Manejo de Ficheros

ÍNDICE

1. Introducción
2. Clases asociadas a ficheros y directorios
3. Flujos
4. Formas de acceso a un fichero
5. Ficheros de Texto
6. Java.io

1. Introducción

Si estás estudiando este módulo, es probable que ya hayas estudiado el de programación, por lo que no te serán desconocidos muchos conceptos que se tratan en este tema.

Ya sabes que cuando apagas el ordenador, los datos de la memoria RAM se pierden. Un ordenador utiliza ficheros para guardar los datos. Piensa que los datos de las canciones que oyes en mp3, las películas que ves en formato avi, o mp4, etc., están, al fin y al cabo, almacenadas en ficheros, los cuales están grabados en un soporte como son los discos duros, DVD, pendrives, etc.

A los datos que se guardan en ficheros se les llama **datos persistentes**, porque persisten más allá de la ejecución de la aplicación que los trata. Los ordenadores almacenan los ficheros en unidades de almacenamiento secundario como discos duros, discos ópticos, etc. En esta unidad veremos, entre otras cosas, cómo hacer con Java las operaciones de crear, actualizar y procesar ficheros.

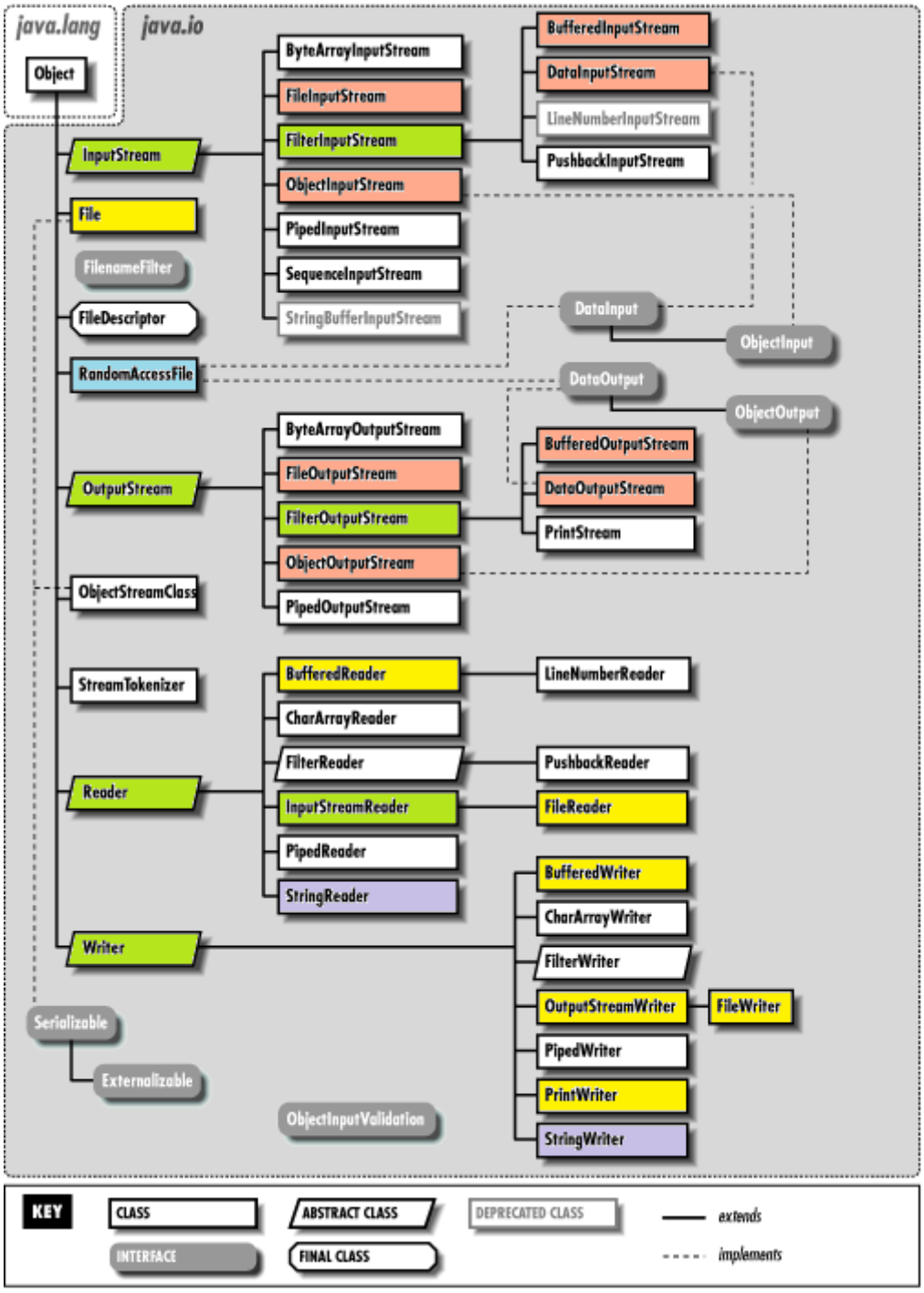
A las operaciones, que constituyen un flujo de información del programa con el exterior, se les conoce como Entrada/Salida (E/S).

Las operaciones de E/S en Java las proporciona el paquete estándar de la API de Java denominado **java.io** que incorpora interfaces, clases y excepciones para acceder a todo tipo de ficheros.

La librería java.io contiene las clases necesarias para gestionar las operaciones de entrada y salida con Java. Estas clases de E/S las podemos agrupar fundamentalmente en:

- Clases para leer entradas desde un flujo de datos.
- Clases para escribir entradas a un flujo de datos.
- Clases para operar con ficheros en el sistema de ficheros local.
- Clases para gestionar la serialización de objetos.

En la siguiente imagen puedes ver las clases de las que se dispone en **java.io**.



Java API Reference – Web Oficial de Oracle
https://docs.oracle.com/en/java/javase/17/docs/api/index.html https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/package-tree.html

Java API Reference – JavaTPoint – I/O
https://www.javatpoint.com/java-io

ESQUEMA DE OBJETOS

Java API Reference – java.io
https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/package-summary.html

Ficheros y estructuras	
File y Files	Acceso a ficheros y carpetas de dispositivos de almacenamiento
Path y Paths	Acceso a ficheros y carpetas de dispositivos de almacenamiento

Ficheros de texto		**UTF8**	**Añadir**
PrintWriter	Clase para escribir en archivos de texto con print	X	
FileWriter	Clase para escribir en archivos de texto que controla **IOExceptions**		
BufferedWriter	Clase para escribir en archivos de texto que controla la escritura a través de **buffer** de forma automática, minimizando la cantidad de escrituras en el dispositivo		
OutputStreamWriter	Clase que usaremos para forzar la escritura en UTF8	X	
FileOutputStream	Clase que usaremos para indicar si se abre para añadir o para escribir		X
FileReader	Clase para leer de archivos de texto que controla IOExceptions		
BufferedReader	Clase para leer de archivos de texto que controla la lectura a través de buffer de forma automática, minimizando la cantidad de lecturas desde el dispositivo		

Ficheros binarios	
FileOutputStream FileInputStream	Clases para escribir o leer bytes de un archivo
DataOutputStream DataInputStream	Clases para escribir o leer de un archivo permitiendo dar formato a los tipos primitivos y objetos String
ObjectOutputStream ObjectInputStream	Clases para escribir o leer de un archivo que permiten conviertir en objetos y variables los vectores de bytes leídos de un InputStream o escritos en un OutputStream.

Ficheros aleatorios	
RandomAccessFile	Clases para trabajar con archivos utilizando acceso aleatorio, es decir, pudiendo posicionarse antes de leer o escribir.

2. Clases asociadas a ficheros y directorios

Hay bastantes métodos involucrados en las clases que en Java nos permiten manipular ficheros y carpetas o directorios.

Vamos a ver primero la clase **File** que nos permite hacer unas cuantas operaciones con ficheros, también veremos cómo filtrar ficheros, o sea, obtener aquellos con una característica determinada, como puede ser que tengan la extensión .odt, o la que nos interese, y por último, en este apartado también veremos como crear y eliminar ficheros y directorios.

2.1 Clase File

La clase **File** proporciona una representación abstracta de ficheros y directorios.

Esta clase, permite examinar y manipular archivos y directorios, independientemente de la plataforma en la que se esté trabajando: Linux, Windows, etc.

Las instancias de la clase File representan nombres de archivo, no los archivos en sí mismos.

El archivo correspondiente a un nombre puede ser que no exista, por esta razón habrá que controlar las posibles excepciones.

Un objeto de clase **File** permite examinar el nombre del archivo, descomponerlo en su rama de directorios o crear el archivo si no existe, pasando el objeto de tipo File a un constructor adecuado como **FileWriter(File f)**, que recibe como parámetro un objeto File.

Para archivos que existen, a través del objeto File, un programa puede examinar los atributos del archivo, cambiar su nombre, borrarlo o cambiar sus permisos. Dado un objeto **File**, podemos hacer las siguientes operaciones con él:

- Renombrar el archivo, con el método **renameTo**(). El objeto File dejará de referirse al archivo renombrado, ya que el String con el nombre del archivo en el objeto File no cambia.
- Borrar el archivo, con el método **delete**(). También, con deleteOnExit() se borra cuando finaliza la ejecución de la máquina virtual Java.
- Crear un nuevo fichero con un nombre único. El método estático **createTempFile**() crea un fichero temporal y devuelve un objeto File que apunta a él. Es útil para crear archivos temporales, que luego se borran, asegurándonos tener un nombre de archivo no repetido.

- Establecer la fecha y la hora de modificación del archivo con **setLastModified**().
  Por ejemplo, se podría hacer:
  new File("prueba.txt").setLastModified(new Date().getTime());
  para establecerle la fecha actual al fichero que se le pasa como parámetro, en este caso prueba.txt.
- Crear un directorio, mediante el método **mkdir**(). También existe mkdirs(), que crea los directorios superiores si no existen.
- Listar el contenido de un directorio: los métodos **list**() y **listFiles**() listan el contenido de un directorio.
  - **list**() devuelve un vector de String con los nombres de los archivos
  - **listFiles**() devuelve un vector de objetos File
- Listar los nombres de archivo de la raíz del sistema de archivos, mediante el método estático **listRoots**().

Mediante la clase File, podemos ver si un fichero cualquiera, digamos por ejemplo texto.txt, existe o no. Para ello, nos valemos del método exists() de la clase File. Hacemos referencia a ese fichero en concreto con el código siguiente:

```
File f = new File("C:/texto.txt");
```

En el siguiente recurso puedes ver una pequeña aplicación en la que se usa File.

Fich01ListarArchivos

```
String carpeta="C:/Windows";

File ruta = new File(carpeta);
if (ruta.exists()) {
      String[] listaArchivos = ruta.list();
      for(int i=0; i<listaArchivos.length; i++){
            System.out.println(listaArchivos[i]);
      }
}
```

Pero, ¿qué pasa si nos interesa copiar un fichero, cómo lo haríamos?

Con la clase File no es suficiente, necesitamos saber más, en concreto, necesitamos hablar de los flujos, como vamos a ver más adelante.

2.2 Rutas de los ficheros

En algunos ejemplos que vemos en el tema, estamos usando la ruta de los ficheros tal y como se usan en MS-DOS, o Windows, es decir, por ejemplo:

```
C:/datos/programacion/fichero.txt
```

Cuando operamos con rutas de ficheros, el carácter separador entre directorios o carpetas suele cambiar dependiendo del sistema operativo en el que se esté ejecutando el programa.

Para evitar problemas en la ejecución de los programas cuando se ejecuten en uno u otro sistema operativo y, por tanto, persiguiendo que nuestras aplicaciones sean lo más portables posibles, se recomienda usar en Java **File.separator** o bien el carácter `'/'`.

Nosotros trabajaremos con rutas relativas al proyecto, es decir:

```
./datos/fichero.txt      o          datos/fichero.txt
```

2.3 Creación y eliminación de ficheros y directorios

Cuando queramos crear un fichero, podemos proceder del siguiente modo:

Fich02Fich1

```
try {
  File fichero = new File("./datos/miFichero.txt");

  if (fichero.createNewFile())
    System.out.println("El fichero se ha creado correctamente");
  else
    System.out.println("No ha podido ser creado el fichero");

} catch (Exception e) {
  e.getMessage();
}
```

Para borrar un fichero podemos usar la clase File, comprobando previamente si existe el fichero, del siguiente modo:

Fich02Fich2

```
File fichero = new File("./datos/miFichero.txt");
if (fichero.exists()) {
  fichero.delete();
}
```

Para crear directorios, podríamos hacer:

Fich02Dir1

```
try {
  boolean exito;
  String directorio = "./datos";
  String varios = "./datos/carpeta1/carpeta2/carpeta3";

  File dir = new File(directorio);

  if (!dir.exists()) {
     exito = (new File(directorio)).mkdir();
     if (exito) {
        System.out.println("Directorio: " + directorio + " creado");
     }
  }

  if (dir.exists()) {
     exito = (new File(varios)).mkdirs();
     if (exito) {
        System.out.println("Directorios: " + varios + " creados");
     }
  }

} catch (Exception e){
  System.err.println("Error: " + e.getMessage());
}
```

Para recorrer un directorio con Java utilizaremos listFiles:

```
File[] ficheros = directorio.listFiles();
```

Fich02Dir2

```
try {
  File dir = new File("./datos");
  File[] ficheros = dir.listFiles();

  for (File file : ficheros) {
     System.out.println(file.getName());
  }

} catch (Exception e){
  System.err.println("Error: " + e.getMessage());
}
```

Para borrar un directorio, tendremos que borrar cada uno de los ficheros y directorios que éste contenga. Al poder almacenar otros directorios, se podría recorrer recursivamente el directorio para ir borrando todos los ficheros.

Si el elemento es un directorio, lo sabemos mediante el método isDirectory().

Fich02Dir3

```
public class Fich02Dir3 {

    public static void main(String[] args) {

        // Crear carpetas y ficheros temporales
        File dirtmp = new File("datos/tmp/files");
        File fichtmp;

        try {
            dirtmp.mkdirs();
            fichtmp = new File("datos/tmp/files/fichero1.txt");
            fichtmp.createNewFile();
            fichtmp = new File("datos/tmp/files/fichero2.txt");
            fichtmp.createNewFile();
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }

        // Mostrar carpetas y ficheros temporales
        File dir = new File("datos/tmp");
        System.out.println("-----------------------------------");
        listDir(dir);

        // Eliminar carpetas y ficheros temporales
        System.out.println("-----------------------------------");
        delDir(dir);
        listDir(dir);
    }

    private static void listDir(File dir) {
        File[] ficheros = dir.listFiles();
        for (File file : ficheros) {
            System.out.println(file.getAbsolutePath()+"\\"+file.getName());
            if (file.isDirectory()) {
                listDir(file);
            }
        }
    }

    private static void delDir(File dir) {
        File[] ficheros = dir.listFiles();
        for (File file : ficheros) {
            if (file.isDirectory()) {
                delDir(file);
            }
            file.delete();
        }
    }

}
```

Java dispone de otra clase para gestionar los archivos y directorios denominada **Path,** con la que se pueden realizar casi las mismas acciones que con **File**.

También se dispone de una clase con métodos **static** adicionales denominada **Paths**, al igual que existe **Files**.

Podremos comprobar que nuevas librerías desarrolladas esperan como parámetro un objeto **Path**, por lo que conviene conocer al menos la posibilidad de conversión.

Fich02Paths

```
try {
  String archivo="./datos/ejemplo.txt";

  File file = new File(archivo);
  Path path = Paths.get(archivo);

  Path pathFromFile = file.toPath();
  File fileFromPath = path.toFile();

  System.out.println("Absolute  Path (File): "
                                + file.getAbsolutePath());
  System.out.println("Canonical Path (File): "
                                + file.getCanonicalPath());
  System.out.println("Absolute  Path (Path): "
                                + path.toAbsolutePath());
  System.out.println("Canonical Path (Path): "
                                + path.toAbsolutePath().normalize());

} catch (IOException ex) {
  System.err.println("Error: " + e.getMessage());
}
```

3. Flujos

Un programa en Java, que necesita realizar una operación de entrada/salida (en adelante E/S), lo hace a través de un **flujo o stream**.

Un flujo es una abstracción de todo aquello que produce o consume información.

La vinculación de este flujo al dispositivo físico la hace el sistema de entrada y salida de Java.

Las clases y métodos de E/S que necesitamos emplear son las mismas independientemente del dispositivo con el que estemos actuando. Luego, el núcleo de Java sabrá si tiene que tratar con el teclado, el monitor, un sistema de archivos o un socket de red; liberando al programador de tener que saber con quién está interactuando.

Java define dos tipos de flujos en el paquete **java.io**:

- **Byte streams (8 bits)**: proporciona lo necesario para la gestión de entradas y salidas de bytes y su uso está orientado a la lectura y escritura de datos binarios. El tratamiento del flujo de bytes viene determinado por dos clases abstractas que son **InputStream** y **OutputStream** Estas dos clases definen los métodos que sus subclases tendrán implementados y, de entre todos, destacan read()y write() que leen y escriben bytes de datos respectivamente.
- **Character streams (16 bits)**: de manera similar a los flujos de bytes, los flujos de caracteres están determinados por dos clases abstractas, en este caso: **Reader** y **Writer**. Dichas clases manejan flujos de caracteres Unicode. Y también de ellas derivan subclases concretas que implementan los métodos definidos en ellas siendo los más destacados los métodos read() y write() que leen y escriben caracteres de datos respectivamente.

4. Formas de acceso a un fichero

En Java se puede utilizar dos **tipos de ficheros (de texto o binarios)** y dos **tipos de acceso a los ficheros (secuencial o aleatorio)**. Si bien, y según la literatura que consultemos, a veces se distingue una tercera forma de acceso denominada concatenación, tuberías o pipes.

- **Acceso secuencial:** En este caso los datos se leen de manera secuencial, desde el comienzo del archivo hasta el final (el cual muchas veces no se conoce a priori). Este es el caso de la lectura del teclado o la escritura en una consola de texto, no se sabe cuándo el operador terminará de escribir.
- **Acceso aleatorio:** los archivos de acceso aleatorio, al igual que lo que sucede usualmente con la memoria (RAM=Random Access Memory), permiten acceder a los datos en forma no secuencial, desordenada. Esto implica que el archivo debe estar disponible en su totalidad al momento de ser accedido, algo que no siempre es posible.
- **Concatenación (tuberías o "pipes"):** Muchas veces es útil hacer conexiones entre programas que se ejecutan simultáneamente dentro de una misma máquina, de modo que lo que uno produce se envía por un "tubo" para ser recibido por el otro, que está esperando a la salida del tubo. Las tuberías cumplen esta función.

5. Ficheros de Texto

5.1 Ficheros de Texto - Escritura

Java incluye muchas clases relacionadas con la entrada y salida en fichero. No veremos todas ellas, sino solo las que según nuestro criterio pueden resultar más útiles y más sencillas.

Un fichero de texto se puede manejar de forma muy similar a la consola, volcando datos línea a línea con "**println**". Una de las formas más simples de conseguirlo en Java es empleando un "**PrintWriter**". Añadiremos la codificación **UTF-8** con la que siempre trabajaremos usando el **Charset** correspondiente.

A diferencia de otros lenguajes como C#, es obligatorio usar un **try-catch** para interceptar los posibles errores:

Fich03PrintWriter1a

```
import java.io.PrintWriter;
import java.io.FileNotFoundException;

public class Fich03PrintWriter1a {
  public static void main(String[] args) {
    try {
      PrintWriter printWriter =
              new PrintWriter("./datos/ejemplo1a.txt", StandardCharsets.UTF_8);
      printWriter.println("Hola!");
      printWriter.close ();
    } catch (FileNotFoundException ex) {
      ex.printStackTrace();
    } catch (IOException ex) {
      ex.printStackTrace();
    }
  }
}
```

Una alternativa menos robusta, pero más compacta, por lo que puede ser útil en fuentes de pequeño tamaño, es no interceptar la excepción, sino permitir que el propio "main" pueda lanzarla, añadiendo "throws", así:

Fich03PrintWriter1b

```
import java.io.PrintWriter;
import java.io.FileNotFoundException;

public class Fich03PrintWriter1b {
  public static void main(String[] args)
                                  throws FileNotFoundException, IOException {
    PrintWriter printWriter =
              new PrintWriter("./datos/ejemplo1b.txt", StandardCharsets.UTF_8);
    printWriter.println ("Hola!");
    printWriter.close ();
  }
}
```

Además, siendo estrictos, puede existir un problema con ese primer programa: si se produce un error durante la escritura, el fichero podría quedar abierto y existir una pequeña fuga de memoria. Para evitarlo, es preferible cerrar el fichero en el bloque final opcional de "**try-catch-finally**".

De paso, podemos interceptar cualquier error de entrada / salida con **IOException**, e importar todo lo relacionado con la entrada y salida haciendo "**import java.io.*;**" en vez de emplear varios "**import**" individuales:

Fich03PrintWriter2

```
public class Fich03PrintWriter2 {
  public static void main(String[] args) {
    PrintWriter printWriter = null;
    try {
      printWriter =
               new PrintWriter("./datos/ejemplo2.txt", StandardCharsets.UTF_8);
      printWriter.println("Hola!");
      printWriter.println("y...");
      printWriter.println("hasta luego!");
    } catch (IOException e) {
      e.printStackTrace();
    } finally {
      if ( printWriter != null ) {
        printWriter.close();
      }
    }
  }
}
```

El constructor habitual de **PrintWriter** destruye el contenido fichero, en caso de que éste existiera. Si, por el contrario, se desea añadir al final de un fichero existente, se debe emplear otro constructor, que no recibirá la ruta del fichero, sino un "**BufferedWriter**", que se apoya en un "**FileWriter**" que tiene "**true**" como segundo parámetro: "**new FileWriter("./datos/ejemplo3.txt", true))**", así:

Fich03PrintWriter3

```
public class Fich03PrintWriter3 {
  public static void main(String[] args) {
    PrintWriter printWriter = null;
    try {
      printWriter = new PrintWriter(
                      new BufferedWriter(
                        new OutputStreamWriter(
                          new FileOutputStream("./datos/ejemplo3.txt", true),
                          StandardCharsets.UTF_8)));
      printWriter.println("Hola otra vez!");
      printWriter.println("y...");
      printWriter.println("hasta luego!");
    } catch (IOException e) {
      e.printStackTrace();
    } finally {
      if ( printWriter != null ) {
        printWriter.close();
      }
    }
  }
}
```

También se podría haber empleado directamente un "**BufferedWriter**" para escribir, sin necesidad de usar la clase "**PrintWriter**", pero se trata de una clase de más bajo nivel, que se parece menos al manejo habitual de la consola, porque no incluye un método "**println**", sino que se debe usar "**write**" para escribir y "**newLine**" para avanzar de línea.

Fich03BufferedWriter1

```
import java.io.*;

public class Fich03BufferedWriter1 {
  public static void main(String[] args) {
    BufferedWriter ficheroSalida = null;
    try {
      ficheroSalida = new BufferedWriter(
          new FileWriter(new File("./datos/ejemplo4.txt")));
      ficheroSalida.write("Línea 1");
      ficheroSalida.newLine();
      ficheroSalida.write("Línea 2");
      ficheroSalida.newLine();
    } catch (IOException e) {
      System.out.println("Ha habido problemas: " + e.getMessage() );
    } finally {
      try {
        if ( ficheroSalida != null ) {
          ficheroSalida.close();
        }
      } catch (IOException e) {
        System.out.println("Ha habido problemas: " + e.getMessage() );
      }
    }
  }
}
```

Otra forma de trabajar con ficheros para evitarnos el finally es usar **try-with-resources**, es decir, indicando los recursos que se cerrarán al terminar el bloque try:

Fich03BufferedWriter2

```
import java.io.*;

public class Fich03BufferedWriter2 {
  public static void main(String[] args) {

    try (BufferedWriter ficheroSalida = new BufferedWriter(
            new FileWriter(new File("./datos/ejemplo4.txt")))) {

      ficheroSalida.write("Línea 1");
      ficheroSalida.newLine();
      ficheroSalida.write("Línea 2");
      ficheroSalida.newLine();

    } catch (IOException e) {
      System.out.println("Ha habido problemas: " + e.getMessage() );
    }

  }
}
```

5.2 Ficheros de Texto - Lectura

Una forma sencilla de leer de un fichero, si nos basta con hacerlo línea a línea, es emplear un "**BufferedReader**", que se apoyará en un "**FileReader**" y que incluye un método "**readLine**" que nos devuelve una cadena de texto (un "**String**").

Si ese String es null, quiere decir que se ha acabado el fichero y no se ha podido leer nada. Por eso, lo habitual es usar un "**while**" (o un "do-while") para leer todo el contenido de un fichero:

Fich04BufferedReader1

```
import java.io.*;

public class Fich04BufferedReader1 {
  public static void main(String[] args) {

    // Primero vemos si el fichero existe
    if (! (new File("./datos/ejemplo3.txt")).exists() ) {
      System.out.println("No he encontrado ejemplo.txt");
      return;
    }

    // En caso de que exista, intentamos leer
    System.out.println("Leyendo fichero...");

    try (BufferedReader ficheroEntrada = new BufferedReader(
             new FileReader(new File("./datos/ejemplo3.txt")))) {

      String linea = null;

      linea = ficheroEntrada.readLine();
      while (linea != null) {
        System.out.println(linea);
        linea = ficheroEntrada.readLine();
      }

    } catch (IOException e) {
      System.out.println( "Ha habido problemas: " + e.getMessage() );
    }

    System.out.println("Fin de la lectura.");
  }
}
```

Es habitual abreviar la lectura y la comprobación, haciéndolas en una misma orden, lo que resulta más compacto pero menos legible:

```
...
  String linea=null;
  while ((linea=ficheroEntrada.readLine()) != null) {
    System.out.println(linea);
  }
...
```

También se podría crear una clase Fichero con los métodos:

- public static void mostrarFichero(String sFich)
- public static void leer(String sFich)
- public static boolean existeFichero(String sFich)

Esto nos permitirá crear una aplicación que utilice esta clase:

Fich04BufferedReader2

```
import java.io.*;

// Clase Fichero
public class Fichero {

   public static void mostrarFichero(String sFich) {
     ...
   }

   public static void leer(String sFich) throws Exception {
     ...
   }

   public static boolean existeFichero(String sFich) throws Exception {
     ...
   }

}

// Programa Principal
public class Fich04BufferedReader2 {
  public static void main(String[] args) {

        System.out.println("LEER FICHERO");
        Fichero.mostrarFichero("./datos/ejemplo3.txt");
        System.out.println("");

        System.out.println("LEER FICHERO");
        Fichero.mostrarFichero("./datos/ejemplo-no-existe.txt");
        System.out.println("");

  }
}
```

5.3 Ficheros de Texto - Resumen

Para los flujos de caracteres, Java dispone de dos clases abstractas: **Reader** y **Writer**.

Si se usan sólo **FileInputStream**, **FileOuputStream**, **FileReader** o **FileWriter**, cada vez que se efectúa una lectura o escritura, se hace físicamente en el disco duro. Si se leen o escriben pocos caracteres cada vez, el proceso se hace costoso y lento por los muchos accesos a disco duro.

Las clases **BufferedReader**, **BufferedWriter,** **BufferedInputStream**, y **BufferedOutputStream** añaden un buffer intermedio. Cuando se lee o escribe, estas clases controla los accesos a disco de forma automática.

Así, si vamos escribiendo, se guardarán los datos en memoria RAM hasta que haya bastantes datos como para hacer una escritura eficiente.

Al leer, la clase leerá más datos de los que se hayan pedido. En las siguientes lecturas nos dará lo que tiene almacenado en memoria RAM, hasta que necesite leer otra vez físicamente. Esta forma de trabajar hace los accesos a disco más eficientes y el programa se ejecuta más rápido.

En el siguiente ejemplo, puedes ver cómo se escribe en un archivo denominado **archivotexto.txt**.

Además, podemos forzar la escritura en disco en cualquier momento con **flush()**.

Fich05EscribirTexto

```
File archivo = new File ("./datos/archivotexto.txt");

// Apertura del fichero y creacion de BufferedWriter
try ( FileWriter fw = new FileWriter (archivo, false);
      BufferedWriter bw = new BufferedWriter(fw); ) {

    // Escritura en el fichero
    bw.write("Este es un fichero de texto");
    bw.newLine();
    bw.write("que contiene varias líneas\n");
    bw.write("y se crea con BufferedWriter.\n");

    // Forzamos la escritura en disco
    bw.flush();

} catch (Exception e) {
    System.out.println(e.getMessage());
}
```

Para crear el fichero y forzar la codificación en UTF8 utilizaremos las clases **FileOuputStream** + **OutputStreamWriter** + **BufferedWriter**.

Además con **FileOutputStream** podemos indicar si abrimos el fichero para sobreescribir o añadir al contenido existente.

Fich05EscribirTextoUTF8 (recomendado)

```
File archivo = new File ("./datos/archivotextoUTF8.txt");

// Apertura del fichero y creación de BufferedWriter
try (BufferedWriter bw = new BufferedWriter(
          new OutputStreamWriter(
            new FileOutputStream(archivo, true), StandardCharsets.UTF_8)) ) {

    // Escritura en el fichero
    bw.write("Este es un fichero de texto");
    bw.newLine();
    bw.write("que contiene varias líneas\n");
    bw.write("y se crea con BufferedWriter.\n");

} catch (Exception e) {
    System.out.println(e.getMessage());
}
```

También podemos abrirlo pero aprovechando los métodos de PrintWriter:

Fich05EscribirTextoUTF8Print (recomendado)

```
File archivo = new File ("./datos/archivotextoUTF8.txt");

// Apertura del fichero y creación de PrintWriter
try (PrintWriter pw = new PrintWriter(
        new BufferedWriter(
          new OutputStreamWriter(
            new FileOutputStream(archivo, true), StandardCharsets.UTF_8)) )) {

    // Escritura en el fichero
    pw.println("Este es un fichero de texto");
    pw.print("que contiene varias líneas\n");
    pw.print("y se crea con BufferedWriter.\n");
    pw.println();

} catch (Exception e) {
    System.out.println(e.getMessage());
}
```

Para especificar la codificación UTF-8 se puede usar la variable **static** de tipo **Charset** indicada en los ejemplos **StandardCharsets.UTF_8** o bien:

```
Charset.forName("UTF-8")
```

En el siguiente ejemplo, puedes ver cómo se lee un archivo guardado denominado **archivotexto.txt**.

Fich05LeerTexto

```
File archivo = new File ("./datos/archivotexto.txt");

// Apertura del fichero y creacion de BufferedReader
try (   FileReader fr = new FileReader(archivo);
        BufferedReader br = new BufferedReader(fr);) {

    // Lectura del fichero
    String linea;
    while ( (linea=br.readLine()) != null) {
        System.out.println(linea);
    }

} catch (Exception e) {
    System.out.println(e.getMessage());
}
```

Utilizando la lectura pero forzando la codificación en UTF8 tendríamos:

Fich05LeerTextoUTF8 (recomendado)

```
File archivo = new File ("./datos/archivotexto.txt");

// Apertura del fichero y creación de BufferedReader
try (BufferedReader br
            = new BufferedReader(
                new InputStreamReader(
                  new FileInputStream(archivo), StandardCharsets.UTF_8)) ) {

    // Lectura del fichero
    String linea;
    while ( (linea=br.readLine()) != null) {
        System.out.println(linea);
    }

} catch (Exception e) {
    System.out.println(e.getMessage());
}
```

Si deseamos leer un fichero de forma completa en RAM (solo para pequeños archivos), podemos usar el constructor de **String** que convierte un conjunto de bytes en un String:

Fich05LeerTextoUTF8Completo (recomendado)

```
try {
   String contenido = new String(
             Files.readAllBytes(Paths.get("./datos/archivotexto.txt")),
             StandardCharsets.UTF_8);
   System.out.println(contenido);
} catch (IOException ex) {
   System.out.println(ex.getMessage());
}
```

Existen de ficheros de configuración que almacenan estructuras de datos en formato texto. Uno de ellos son los de propiedades.

Java dispone de una clase **Properties** que almacena en RAM pares de valores, es decir, clave y valor. Además, dispone de un método que graba toda la información en un fichero de texto.

Fich06EscribirProperties

```
// Crear Fichero
Properties props = new Properties();
FileOutputStream fos = new FileOutputStream("./datos/usuario.properties");

// Almacenar datos
    // props.setProperty() es equivalente a properties.put()
props.setProperty("usuario", "juan");
props.setProperty("email", "juan@dominio.com");

// Escribir las propiedades en el fichero
props.store(fos, "Fichero de propiedades generado desde programa Java");

// Cerrar fichero
fos.close();
```

De forma similar, podemos recuperar los datos:

Fich06LeerProperties

```
    // Preparar Variables
    Properties properties = new Properties();
    FileInputStream fis = new FileInputStream("./datos/usuario.properties");

    // Leer datos
    properties.load(fis);

    // Cerrar fichero
    fis.close();


    // Mostrar todas las propiedades
    System.out.println("Properties......: " + properties.toString());

    // Mostrar todas las claves
    System.out.println("Claves..........: " + properties.keySet());

    // Mostrar valor de una clave
    System.out.println("Valor de usuario: " +
                                    properties.getProperty("usuario"));

    // Recorrer todas las propiedades
    System.out.println("------------------");
    properties.forEach((k, v) -> {
        System.out.println("Clave: " + k);
        System.out.println("Valor: " + v);
        System.out.println("------------------");
    });
```

Algunas librerías están diseñadas para trabajar con archivos, por lo que esperan un objeto **Reader** o **Writer**, lo que nos obliga a trabajar con un sistema de almacenamiento físico como el disco duro.

Para trabajar con archivos pero en memoria RAM disponemos de las clases **StringReader** y **StringWriter**, lo que nos permite realizar las mismas acciones que hemos visto en RAM. Además, tenemos la posibilidad de convertir fácilmente su contenido en un objeto String.

Fich06StringWriter

```
try (
   StringWriter sw = new StringWriter();
   PrintWriter pw = new PrintWriter(sw)) {

   // Escritura en el fichero con PrintWriter
   pw.println("Este es un fichero de texto");
   pw.print("que contiene varias líneas\n");
   pw.print("y se crea con BufferedWriter.\n");
   pw.println();

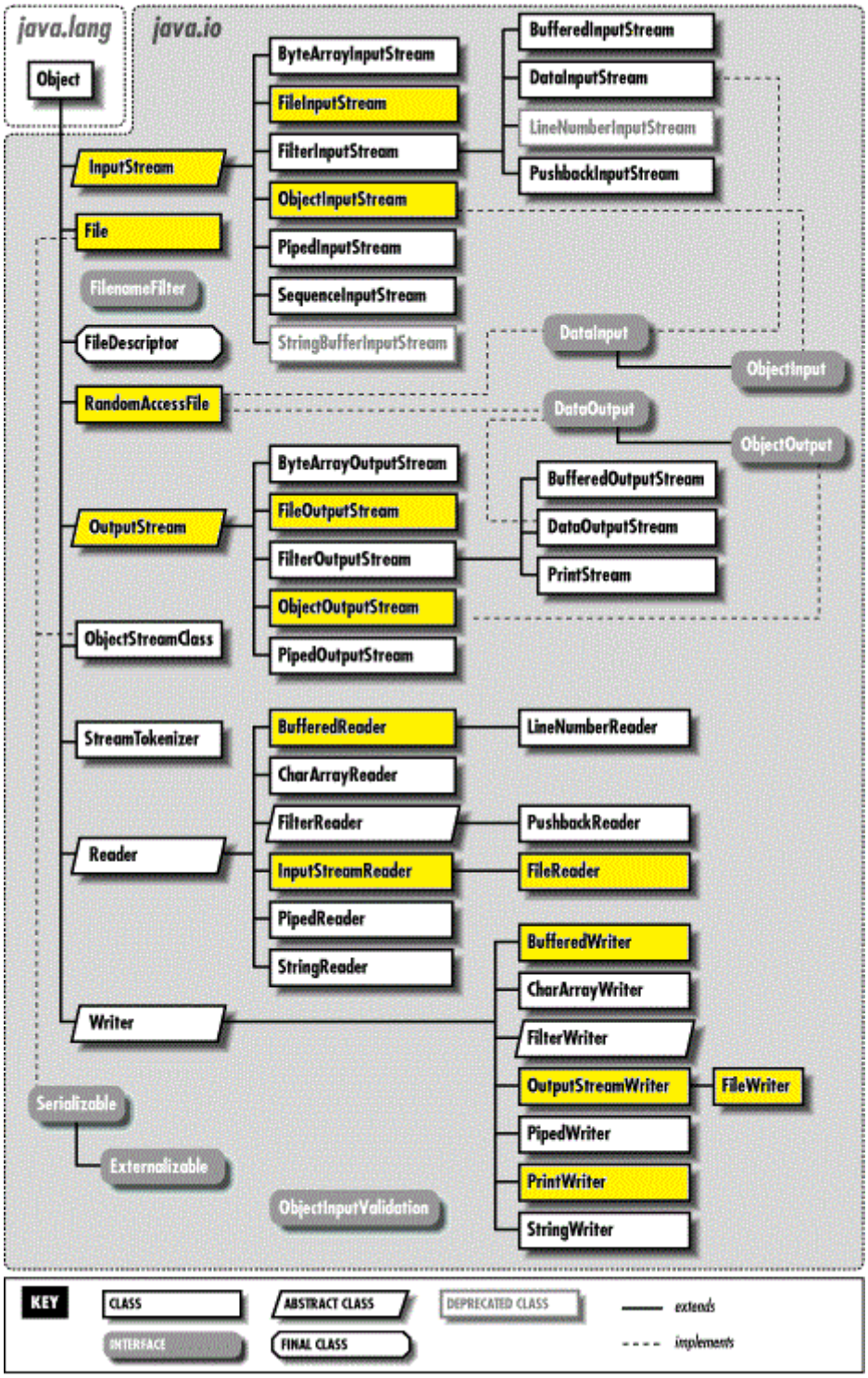
   // Escritura en el fichero con StringWriter
   sw.write("Con write se puede escribir directamente en el StringWriter\n");
   sw.write("Incluso varias líneas.\n");

   System.out.println(sw.toString());

} catch (IOException ex) {
   System.out.println(ex.getMessage());
}
```

6. Java.io

Además de las clases usadas en los ejemplos, existen otras para manejar archivos binarios. Las más usadas de Java.io son:



Para ver más ejemplos
https://www.javatpoint.com/java-io